

Tutorial: Fault Injection with ChipWhisperer Lite (Password)

Ryan Evans, Logan Reichling, and Boyang Wang

Data and System Security Lab (DaSec), ECE Department, University of Cincinnati

Last modified: August 2026, Status: Completed

1 Overview

This document provides and explains the setup and workflow required in order to demonstrate fault injection attacks on a software implementation of password checker program using ChipWhisperer Lite. The fault injection attack, if successful, can alter the execution of the program and bypass the password verification even if the provided password is not correct. Fault injections in this document are introduced through clock glitching.

This document is primarily based on the fault injection notebook¹ from ChipWhisperer website. We made some minor customized changes.

Acknowledgments. The research and education efforts associated with this document are supported by CHEST, the NSF IUCRC Center for Hardware and Embedded Systems Security and Trust.

2 Software and Hardware Settings

In this document, we use the following hardware and software.

- Hardware: ChipWhisperer Lite with an ARM STM32F3 microcontroller as a target, USB cable (both available from CW Lite).
- Operating System: Ubuntu 22.04 (or a VM with Ubuntu 22.04), several Linux packages, Python packages, and our customized jupyter notebook on GitHub repository ²

Note: Skip Section 3 and Section 4.1 if we have done it in other CW Lite tutorials.

3 ChipWhisperer Environment Setup

3.1 ChipWhisperer Overview

ChipWhisperer is an open source toolchain dedicated to hardware security research. This toolchain consists of several layers of open source components, including hardware, firmware, and

¹ https://github.com/newaetech/chipwhisperer/jupyter/courses/fault101/Fault_1.1_Introduction_to_Clock_Glitching.ipynb

² https://github.com/UCdasec/Tutorial-CW/main/Setup_Scripts/Setup_CW_Lite_Fault_Password.ipynb

software. More information regarding ChipWhisperer can be found on ChipWhisperer's GitHub repository³ and official website⁴.

There are multiple ChipWhisperer hardware available on the market. In this tutorial document, we will use a ChipWhisperer Lite (with an ARM STM32F3 microcontroller as a target). An picture of a CW Lite (with an AVR XMEGA microcontroller as a target) from ChipWhisperer website⁵ is presented in Fig. 1. The CW main board (on the left side of the picture) is in charge of communicating with the target and capturing power traces from the target board. The target board (on the right side of the picture) is where we can upload a binary (e.g., a binary of AES encryption program) and execute it on the target microcontroller.

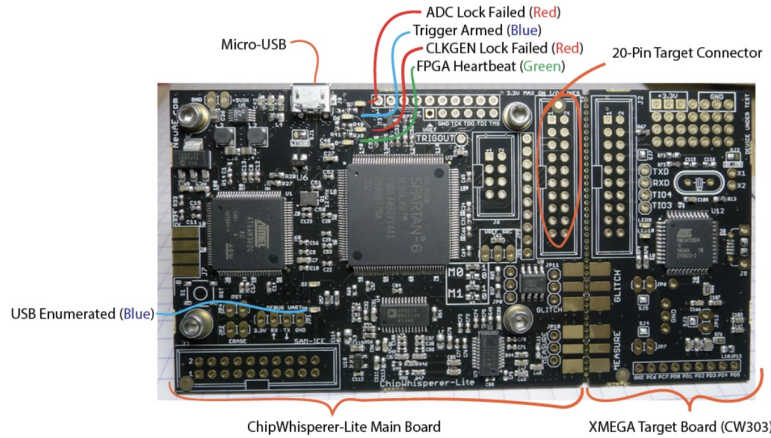


Fig. 1: CW Lite with an AVR XMEGA microcontroller as a target

ChipWhisperer Lite (with an ARM STM32F3 microcontroller as a target) shares the same main board and same layout of the target board but with an ARM STM32F3 microcontroller as a target. A picture of it can be found below.

3.2 ChipWhisperer Installation

Before we connect CW Lite hardware to our computer, we first need to install the ChipWhisperer environment. Specifically, we can follow the instructions and steps from this ChipWhisperer webpage⁶ to install the ChipWhisperer environment. We assume that we are using a computer running Ubuntu or a VM running Ubuntu. We can either choose (1) *the quick installation option* by running all the commands at once or *the manual installation option* by running these commands step by step mentioned on the webpage. We recommend the manual installation option. Specifically,

³ <https://github.com/newaetech/chipwhisperer>

⁴ <https://chipwhisperer.readthedocs.io/en/latest/getting-started.html>

⁵ <https://chipwhisperer.readthedocs.io/en/latest/getting-started.html>

⁶ <https://chipwhisperer.readthedocs.io/en/latest/linux-install.html>

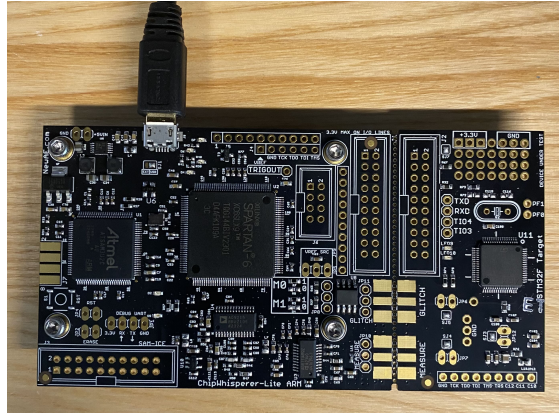


Fig. 2: CW Lite with an ARM STM32F3 microcontroller as a target

we can start from the block below and continue all the remaining steps until we reach the end of the webpage. The entire process takes **8~15 minutes** depending on the system. We also provide a short video⁷ walking through each step during the manual installation.

- Introduction
- Getting Started
- Overview
- Installation**
- Windows Installation
- Windows Drivers
- Linux Installation
- Mac OS X Installation
- Virtual Machine Installation
- Running ChipWhisperer
- Updating ChipWhisperer
- Support

Manual Installation

Required Packages

Begin by updating all your packages:

```
sudo apt update && sudo apt upgrade
```

Next, grab the prerequisites for building firmware for targets, as well as python:

```
sudo apt install libusb-dev make git avr-libc gcc-avr \
gcc-arm-none-eabi libusb-1.0-0-dev usbutils
```

Fig. 3: Manual installation for ChipWhisperer (from ChipWhisperer website)

Make sure we reboot the system as instructed during the process. At the end of the installation, we should be able to enable the ChipWhisperer environment and run jupyter notebook successfully. Specifically, when we run

```
1 source ~/.cwvenv/bin/activate
```

We will see the “**(.cwvenv)**” to the left of working directory if the environment is installed successfully

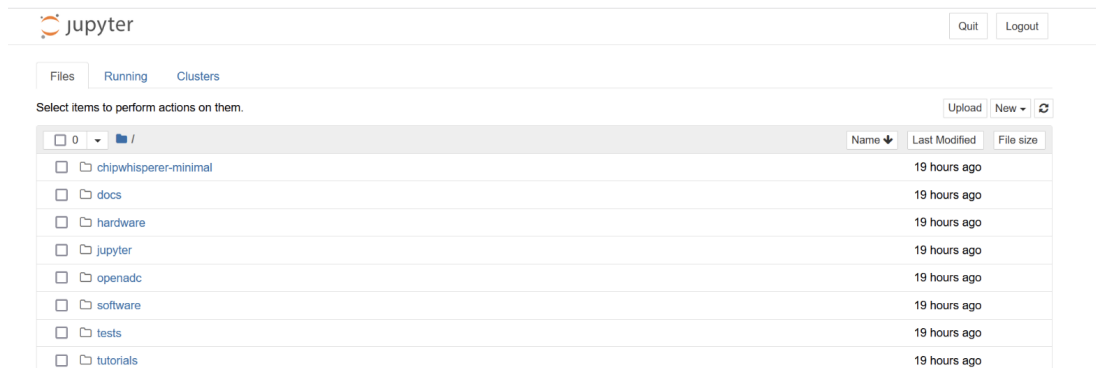
```
dung-dang@dungdang-ThinkPad-T15p-Gen-3:~$ source ~/.cwvenv/bin/activate
(.cwvenv) dung-dang@dungdang-ThinkPad-T15p-Gen-3:~$
```

⁷ https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Installation/1_zytbs228

Next, we can open the jupyter lab by running the command below in the terminal

```
1 jupyter lab
```

The Jupyter server will begin to boot. Wait until the Jupyter lab automatically opens in the web browser and we should be able to see similar as shown below.



When we start Jupyter Lab, it runs as a foreground process and takes over that terminal window to keep the server alive. This blocks the current terminal session, preventing us from executing additional commands in that specific window. If we need to run other commands, we can open a new terminal tab or press “Ctrl” + “C” then press “Y” to exit the Jupyter lab.

3.3 Connect and Verify Hardware

Now, we can connect ChipWhisperer Lite to our computer by using the USB cable (offered by CW Lite box). The computer should be able to automatically recognize the CW Lite. If the connection is successful, we should observe the blue light on the board is ON and flashes (as shown in Fig. 7).

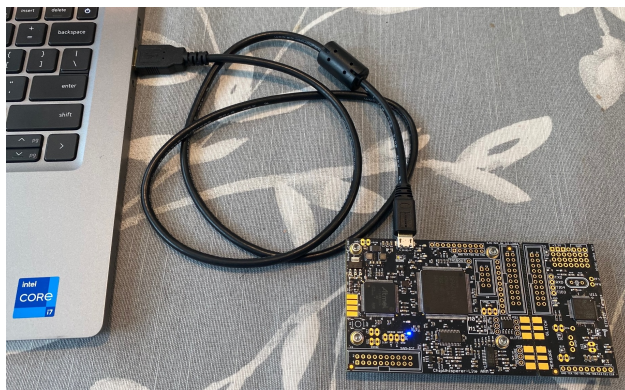


Fig. 4: CW Lite is connected to a computer (blue light flashes)



Fig. 5: Dropdown menu of VM

If we use a VM, make sure we attach the ChipWhisperer USB to the VM through the USB dropdown (as shown in Fig. 5)

Next, run the following command to enable ChipWhisperer environment

```
1 source ~/.cwvenv/bin/activate
2 jupyter lab
```

In Jupyter notebook window, click and choose `chipwhisperer` $\Rightarrow$ `jupyter` $\Rightarrow$ `Setup_Scripts`, and open `Setup_Generic.ipynb`.

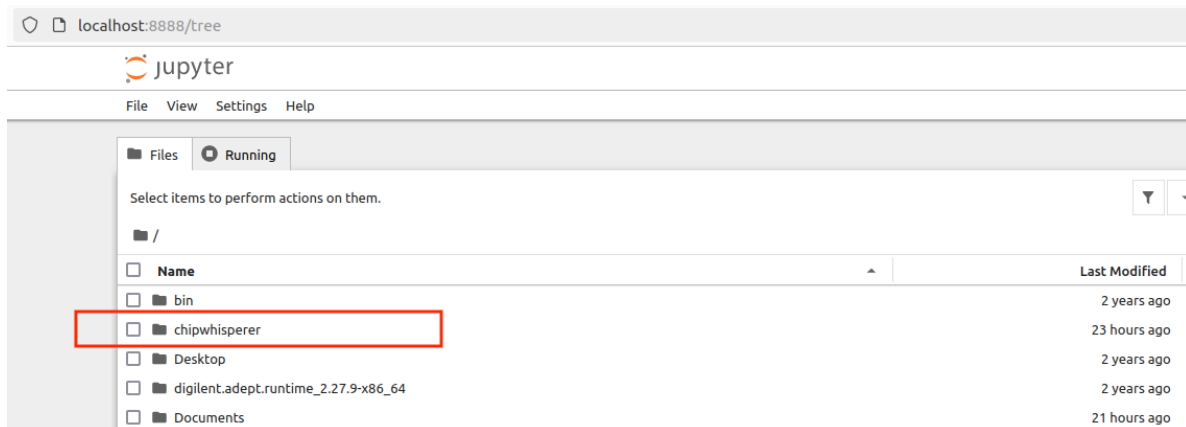


Fig. 6: Jupyter notebook (`./chipwhisperer`)

Execute the first code block of `Setup_Generic.ipynb` by clicking “Run” as shown in Fig. 8. After 1 or 2 seconds, we should observe it outputs `INFO: Found ChipWhisperer`, which indicates our computer recognize the CW Lite hardware successfully. At the same time, we should also observe that an additional green light is ON and flashes on the board. We also provide a short video⁸ walking through each step during this hardware connection testing process.

⁸ https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Connection_Verify/1_800go4yr

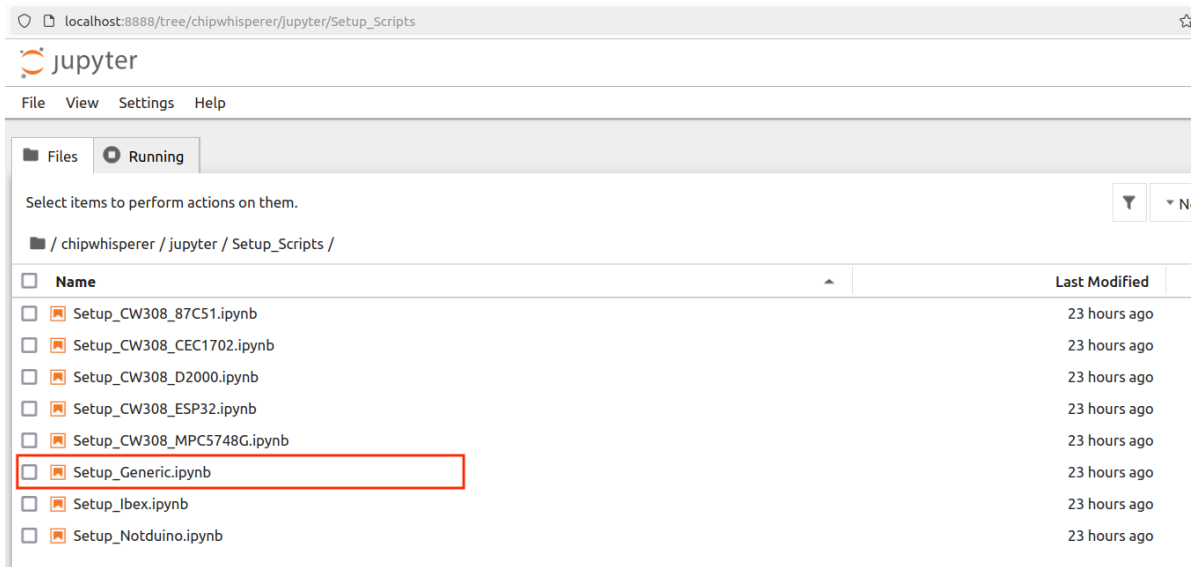


Fig. 7: Jupyter notebook (./chipwhisperer/jupyter/Setup_Scripts)



Fig. 8: Run the first code block of Setup_Generic.ipynb

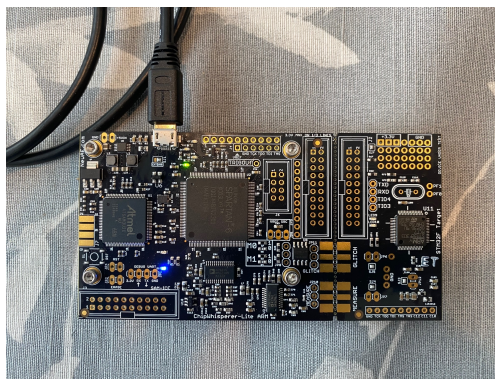


Fig. 9: CW Lite is recognized by the computer (green light flashes)

4 Fault Injection on Password Checker Program

4.1 Install Additional Libraries and Clone Our Repository

Assuming we have installed ChipWhisperer environment, connect and verify the connection to CW Lite hardware successfully as described above, we now install a few additional libraries needed for our customized jupyter notebook and download it.

- Close and exit the previous jupyter sessions (Ctrl + C)
- Ensure that CW Lite hardware is connected
- Enable the CW environment is enabled by running the following command again

```
1 source ~/.cwenv/bin/activate
```

- Install the following Python packages with the following command. These packages are needed for our customized jupyter notebook.

```
1 pip install ipython scikit-learn PyCryptodomex tqdm reportlab h5py
```

- Ensure we are working in the home directory by typing

```
1 cd ~/
```

- Clone our GitHub repository⁹ by running the following command

```
1 git clone https://github.com/UCdasec/Tutorial-CW.git
```

4.2 Password Checker Program

We use a password checker program, which is part of the C code `simpleserial-glitch.c` provided by ChipWhisperer. If we have install chipwhisperer already in `/home/username`, a copy of this implementation can be found here¹⁰. We also present the program below

```
1 // simpleserial-glitch.c
2 // .....
3 #if SS_VER == SS_VER_2_1
4 uint8_t password(uint8_t cmd, uint8_t scmd, uint8_t len, uint8_t* pw)
5 #else
6 uint8_t password(uint8_t* pw, uint8_t len)
7 #endif
8 {
9     char passwd[] = "touch";
10    char passok = 1;
11    int cnt;
12
```

⁹ <https://github.com/UCdasec/Tutorial-CW>

¹⁰ `home/username/chipwhisperer/firmware/mcu/simpleserial-glitch/simpleserial-glitch.c`

```
13     trigger_high();
14
15     //Simple test - doesn't check for too-long password!
16     for(cnt = 0; cnt < 5; cnt++){
17         if (pw[cnt] != passwd[cnt]){
18             passok = 0;
19         }
20     }
21
22     trigger_low();
23
24     simpleserial_put('r', 1, (uint8_t*)&passok);
25     return 0x00;
26 }
```

As we can see from the above program, the real password (**touch**) is hard-coded in the program for the ease of demonstration. If the given password from a user is correct, the variable **passok** remains 1. Otherwise, **passok** will be updated to 0. ChipWhisperer's simple serial protocol will retrieve/read the value of **passok** and send it back to the computer connect to CW Lite by calling `simpleserial_put('r', 1, (uint8_t*)&passok)` at the end.

Given incorrect characters in the password provided, if glitching occurs at the time when **passok** = 0 (at line 18), we can prevent the program from updating the value of **passok** from 1 to 0, which will keep the value of **passok** to be 1 and allow the incorrect password to pass the verification.

It is also worth mention that setting the default value of **passok** as 1 (i.e., considering a given password is correct by default) in this example is not a good idea from security perspective. However, this default example is good enough for us to explore the concept and impacts of fault injection with glitching.

We can also modify or customize the C code of this password checker program if necessary. In this tutorial, we use the default one from ChipWhisperer without any modifications.

4.3 Run Fault Injection

To run the fault injection on this password checker program on ChipWhisperer Lite, we can follow the steps below. We also provide a short video¹¹ walking through each step for the fault injection.

1. Open Terminal and run the command below to enable the environment to enable the Jupyter Notebook if needed

```
1 source ~/.cwenv/bin/activate
2 jupyter lab
```

A web browser tab for jupyter notebook will open.

¹¹ https://ceas.mediaspace.kaltura.com/media/Tutorial_CW_Lite_Fault_Password/1_42hk9jcm

2. Connect ChipWhisperer Lite to our computer and ensure only one blue light is ON and flashes (if not, gently disconnect the USB cable and reconnect). Otherwise, we may get `scope() not defined` error later on.
3. In the web browser for Jupyter notebook, direct to “/Tutorial-CW/Setup_Scripts”, then click on the “**Setup_CW_Lite_Fault_Password.ipynb**”. This script contains multiple code cells to initialize the settings, compile and load a binary to the target board, and perform fault injection with clock glitching while the binary is running on the target board. We will explain each cell one by one below.
4. Before we run any cells in this notebook, we can restart the kernel and clean all the outputs from previous runs.

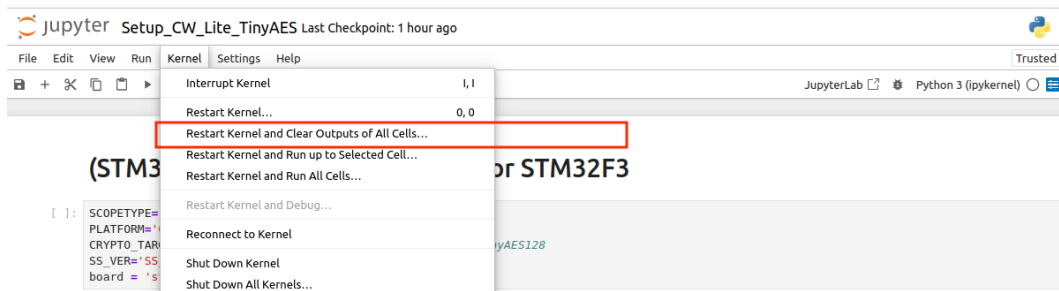


Fig. 10: Restart kernel and clean all the outputs

5. Run the first cell to enable STM32F3 as the target board. Make sure the SCOPETYPE is OPENADC and PLATFORM is CWLITEARM. To run an individual cell, select the cell by left-clicking on it, then press the “Run” button

(STM32F) Run for STM32F3

```
In [1]: SCOPETYPE = 'OPENADC'
        #PLATFORM='CW308_STM32F3'
        PLATFORM = 'CWLITEARM' # OR 'CWLITEXMEGA'
        SS_VER = 'SS_VER_2_1'
```

Fig. 11: Choose the target board setting (Run the STM32F one only)

6. Next, we can run the **first cell** from “Detect, Compile, and Upload to ChipWhisperer” to detect the CW Lite hardware. If successful, we should see the message `INFO: Found ChipWhisperer` again along with some scope information.

Fig. 12: Detect CW Lite

7. Next, we run the **second cell** from “Detect, Compile, and Upload to ChipWhisperer” to compile the C code of password checker program. We should observe the binary of the program, named (simpleserial-glitch-CWLITEARM.elf), is generated successfully.

```
In [3]: %%bash -s "$PLATFORM" "$SS_VER"
#cd ../../../../firmware/mcu/simpleserial-glitch
# change the path if needed
cd /home/$USER/chipwhisperer/firmware/mcu/simpleserial-glitch
make PLATFORM=$1 CRYPTO_TARGET=NONE SS_VER=$2 -j

SS_VER set to SS_VER_2_1
SS_VER set to SS_VER_2_1
```

Fig. 13: Compile the C code of Password Checker

```
arm-none-eabi-nm -n simpleserial-glitch-CWLITEARM.elf > simpleserial-glitch-CWLITEARM.sym
Size after:
      text      data       bss       dec       hex filename
      5860         8      1560      7428      1d04 simpleserial-glitch-CWLITEARM.elf

+-----+
+ Default target does full rebuild each time.
+ Specify buildtarget == allquick == to avoid full rebuild
+-----+
+-----+
+ Built for platform CW-Lite Arm \STM32F3\ with:
+ CRYPTO_TARGET = NONE
+ CRYPTO_OPTIONS =
+-----+
```

Fig. 14: Compile the C code of Password Checker (Expected Output)

8. Next, we run the **third cell** from “Detect, Compile, and Upload to ChipWhisperer” to flash the binary to the target board. We should see the message saying “Verified flask OK” for the binary file in hex format. Make sure we update the username in the file path based on our system setting.

```
In [4]: from pathlib import Path

# Define the path of our firmware and macke sure it exists
# path = "../../firmware/mcu/PASS_CHECK_STRNCMP/simpleserial-glitch-{}.hex".format(PLATFORM)
# change the path if needed
path = "/home/boyang/chipwhisperer/firmware/mcu/simpleserial-glitch/simpleserial-glitch-{}.hex".format(PLATFORM)

if not Path(path).exists():
```

Fig. 15: Flash the binary to target board

```
Program path: /home/boyang/chipwhisperer/firmware/mcu/simpleserial-glitch/simpleserial-glitch-CWLITEARM.hex
Detected known STM32F3: STM32F302xB(C)/303xB(C)
Extended erase (0x44), this can take ten seconds or more
Attempting to program 5867 bytes at 0x8000000
STM32F Programming flash...
STM32F Reading flash...
Verified flash OK, 5867 bytes
```

Fig. 16: Flash the binary to target board (Expected output)

- Next, we run the next cell (“Step 1b”) to make sure that we can reset the device. This is important for fault injection as most of the faults will lead to errors and will crash/halt the device. Being able to reset the device will allow us to continue to run the fault analysis without stopping the device.

Step 1b - Make sure we can reboot when we break things :D

When we run our attacks, we are likely going to **crash the device**.

Well, when that happens all we have to do is reboot it. If you are wanting a workout, we can unplug and plug in the device every crash, but lets be lazy and automate this.

Function `reboot_flush()` defined below will reset the device for us.

```
In [5]: if PLATFORM == "CWLITEXMEGA":  
        def reboot_flush():  
            scope.io.pdic = False  
            time.sleep(0.2)
```

Fig. 17: Test reset function

- Next, we run cell “Step 2 - Communicate with Device” to send a password to the binary on the microcontroller, get a response back from it, and print out the response. Specifically, we provide the first argument as “p” to enable the password check function (there are other functions in the binary, each with a different character to activate) and provide an incorrect password with one byte (0x00) in the `target.simpleserial.write` function. We should observe information with VALID RESPONSE. We are particularly interested in **Board Responds** with provide one byte 00, which is the return of the binary from the target indicating the provided password (i.e., 0x00) is incorrect.

Step 2 - Communicate with the Device

We will communicate using **serial** comms. The device will be listening for us, and we will be listening for the device AFTER we first send a message.

Well, because we are likely crashing the device, sometimes we hear *nothing* back. The function `simpleserial_read_witherrors` will allow us to listen for a short time, but then assume the device crashed with we don't hear back.

```
:
# Clear the device real quick
reboot_flush()

# WRITE to the serial line a "p"
# Passing "p" will enable the password program
# Passing a test password with one char 0x00
target.simpleserial_write("p", bytearray([0x00]))
```

Fig. 18: Communicate with the binary on the target

```
VALID RESPONSE
=====
Board Responds: CWbytearray(b'00')
Raw Response: CWbytearray(b'00 72 01 00 99 00')
Error Code?: bytearray(b'\x00')
```

Fig. 19: Communicate with the binary on the target (Expected Output)

11. Next, we can run the next cell “Step 3b - Sending a wrong password” to repeat the step above but with another incorrect password with five bytes (0x00 00 00 00 00).

Step 3b - Sending a wrong password

```
In [ ]: reboot_flush()

# We'll send all 00's as the password, and see what the device has to say
password = bytearray([0x00]*5)

# Write the password to the serial.
target.simpleserial_write('p', password)

val = target.simpleserial_read_witherrors('r', 1, glitch_timeout=10)#For loop check
```

Fig. 20: Sending a wrong password

```
VALID RETURN
=====
The raw response is: {'valid': True, 'payload': CWbytearray(b'00'), 'full_response': CWbytearray(b'00
72 01 00 99 00'), 'rv': bytearray(b'\x00')}
PASSWORD ACCEPTED? False
```

Fig. 21: Sending a wrong password (Expected output, CWbytearray with b'00')

We should expect the same response with a byte **0x00** from the target as the response from the last cell.

12. Next, we can run the next cell “Step 3c - Sending the correct password” to pass the correct password (0x74 6F 75 63 68, which is “touch”) to the binary

Step 3c - Sending the correct password

```

: # Flush the device
  reboot_flush()

# Write the password "touch" in ASCII to the device
pw = bytearray([0x74, 0x6F, 0x75, 0x63, 0x68])
target.simpleserial_write('p', pw)

# Attempt to read a response from the device
val = target.simpleserial_read_witherrors('r', 1, glitch_timeout=10)#For loop check

```

Fig. 22: Sending the correct password

```

VALID RETURN
=====
The raw response is: {'valid': True, 'payload': CWbytearray(b'01'), 'full_response': CWbytearray(b'00
72 01 01 d4 00'), 'rv': bytearray(b'\x00')}
PASSWORD ACCEPTED? True

```

Fig. 23: Sending the correct password (Expected Results, CWbytearray with b'01')

We should expect the response with a byte **0x01** from the target indicating the password is correct.

- Next, we run the two cells from “Step 4 - Fault Campaign; Time to bypass the password” to setup the plots for fault campaign, where these plots are empty/blank initially and will show the results in real time once we start the fault campaign later.

We also need to tell it what glitch parameters we want to scan through, in this case width and offset:

```

In [ ]: gc = cw.GlitchController(groups=["success", "reset", "normal"], parameters=["width", "offset", "tries"]
        gc.display_stats()

In [ ]: gc.glitch_plot(plotdots={"success":"+g", "reset":"+xr"}, "#", "normal":'xb'})

```

Fig. 24: Setup the fault campaign

```

gc = cw.GlitchController(groups=["success", "reset", "normal"], parameters=["width", "offset", "tries"])
gc.display_stats()

```

success co...	0
reset count:	0
normal cou...	0
width setti...	0.0
offset setti...	0.0
tries setting:	0.0

```

gc.glitch_plot(plotdots={"success":"+g", "reset":"+xr"}, "#", "normal":'xb'})

```

Fig. 25: Setup the fault campaign (Expected Results , part 1)

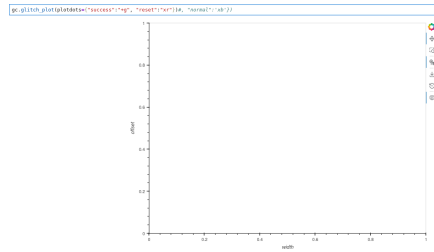


Fig. 26: Setup the fault campaign (Expected Results , part 2)

14. Next, we can run “Run the Campaign!” cell to start the fault campaign. **Before we run**, we can review parameters, including `scope.glitch.ext_offset` (8, by default), `width` (1 to 10, by default), `offset` (-4 to 4, by default), and `scope.glitch.repeat` (5, by default). These parameters defined the range of the fault campaign and how many times we would like to run for each combination of width, offset and ext_offset. Given the same combination of width, offset and ext_offset, even if we find the right moment, not every time the fault will be successful. That is why trying it multiple times could be helpful. These default values provided in the notebook would work fine with this example, where we can find multiple faults successfully. It will take about **10~15 minutes** to complete the fault campaign (depending on the system we use). If needed, we can also change these parameters as well (a wider range and a higher value of repeat times lead to longer fault campaign time).

4b - Run the Campagin!

Below we define the function `brute_force`. This function is iteratively sweep thousands of combinations of `width`, `offset` and `ext_offset`.

By doing so, we should find a few combinations that craft a glitch just right - one that skips instructions in the password check function allowing us to bypass the check!

```
import chipwhisperer.common.results.glitch as glitch
from tqdm.notebook import trange
import struct
```

Fig. 27: Run the fault campaign

```
scope.glitch.ext_offset = 8

def brute_force(gc, width: tuple[int,int] = (1,10), offset: tuple[int,int] = (-4,4), ext_offset: tuple[int,int] = (0,8)):
    gc.set_range('width', width[0], width[1])
    gc.set_range('offset', offset[0], offset[1])
    #gc.set_range('ext_offset', ext_offset[0], ext_offset[1])
    gc.set_global_step([16,8,4,2,1])

    step = 1

    scope.glitch.repeat = 5
    reboot_flush()
```

Fig. 28: Run the fault campaign (reviewing parameters)

In addition, we also want to make sure that we still pass an incorrect password. We will also

```
# Arm the scope, write the password, and capture
scope.arm()
target.simpleserial_write('p', bytearray([0x00]*5)) #send an incorrect password
ret = scope.capture()
```

Fig. 29: Run the fault campaign (incorrect password, 0x00 00 00 00 00)

need to define what is considered as “success”, “normal”, and “reset” based on the response we obtain from each execution of the binary on the target. In this example, if the return of the binary is timeout or invalid, it is considered as a reset. If it has a valid return, and its return code `retcode` is 0, then it is considered as a normal. If it has a valid return, and its return code `retcode` is 1, then it is considered as a “success” (i.e., the binary passes the verification even if the password is incorrect).

```
if ret:
    print('Timeout - no trigger')
    gc.add("reset")
    reboot_flush()
else:
    # If the return is not valid, the device was reset
    if not val['valid']:
        gc.add("reset")
    else:
        # IF we do have a valid return, it should be between 0 and 1
        retcode = int(val['payload'].hex(), 16)
        if retcode == 1:
            # Correct! An incorrect Password was accepted!
            gc.add("success")
            print(val['payload'])
            print(scope.glitch.width, scope.glitch.offset, scope.glitch.ext_offset)
        elif retcode == 0:
            # An incorrect password was denied! This is to be expected.
            gc.add("normal")
brute_force(gc)
```

Fig. 30: Define success, normal, and reset

After we review everything, we can run the cell to start the campaign.

15. Once the fault campaign starts, we should observe that the campaign goes through all the combinations of parameters one by one and update results live in the two plots we generated early. We can first see in the bar plot, in which the number of success, reset, and normal will increase. Most of these will be reset and only a few will be success. ***However, as long as we can find 1 success, it shows us that we can trick the program.*** The actual number of success we can find varies depending on the system (be patient and we will find some success!). In the x-y plot, we can also observe the markers for the result of each execution. A star ★ marker indicates a success and a x marker suggests a reset or normal. We can also see that even if with

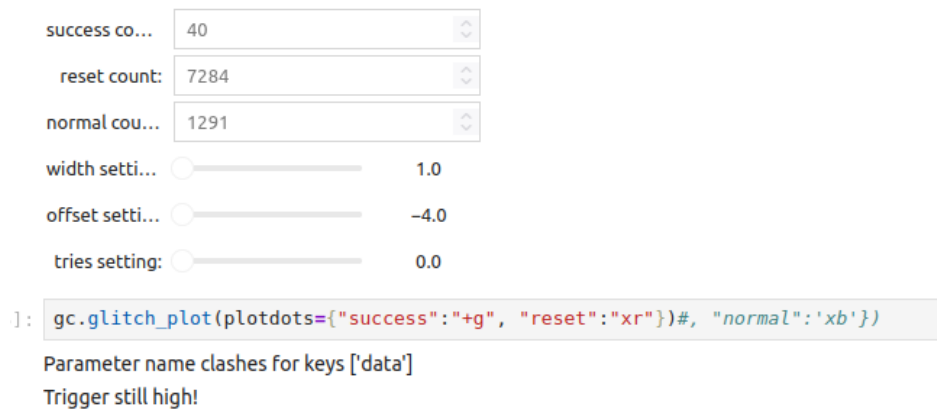


Fig. 31: Expected results (part 1) from fault campaign

the same combination, not every time will be a success. However, we can see there are few combinations leading more success.

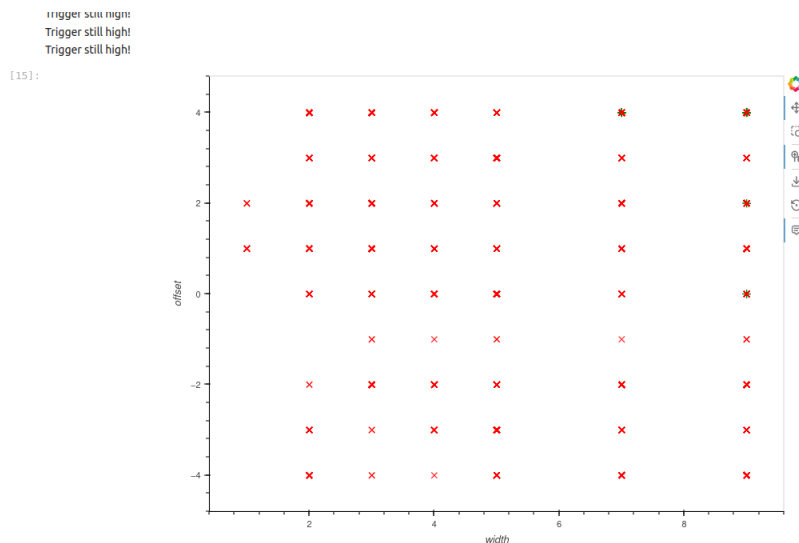


Fig. 32: Expected results (part 2) from fault campaign

16. **Result Summary.** As we can see from this example fault campaign, a total number of **8,615** executions of the target program with the given incorrect password were performed on the target board. Among those, **1,291 are normal** (i.e., the program still indicates the given password is incorrect), **7,284 are reset** (i.e., the program was interrupted and could not return results properly), **40 are success** (i.e., the program falsely believes the given password is the same as the correct password) due to the impact of clock clitching.
17. There are additional blocks we can run after the campaign to further calculate statistic information. We can also try it as well. This is the end of the fault campaign.

4.4 Additional Fun Activities (Try Yourself!)

There are a couple of fun activities that we can do to further enhance our understanding of fault injection with this example. We provide some general information here without specific details. Each is a separate activity based on the example we have showed so far.

- **Activity 1.** There are certain combinations of width and offset (`ext_offset` is fixed as 8 in this example), leading to a higher number of successes. Please find one of the combinations, e.g., `width = 9` and `offset = 4`, in the outputs included in the notebook, reset width and offset as these specific values only but increase the number of tries on this combination, e.g., `scope.glitch.repeat = 200`. Run the fault campaign again on these specific values only rather than going through the entire search space we run previously. Compare the results to see if we have a higher success rate than before.
- **Activity 2.** Find the source C code by following the path¹², and update the hard-coded password ‘touch’ to a different password with 5 characters, rerun the fault campaign starting from ‘Detect, Compile, and Flash to ChipWhisperer’ again and compare the results to see if we have a (significant) higher number of successes.

5 Conclusion

This concludes this tutorial.

¹² `home/username/chipwhisperer/firmware/mcu/simpleserial-glitch/simpleserial-glitch.c`